Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «КиСБ»
Тема: «Избыточное кодирование данных в информационных системах»

Выполнил:
Студент 4-го курса
Группы АС-66
Езепчук А.С.
Проверил:
Хацкевич А.С.

Брест 2026

Цель работы: приобрести практические навыки кодирования декодирования двоичных данных.

Вариант 3

Задание:

1. Разработать приложение для кодирования/декодирования двоичной информации кодом Хемминга с минимальным кодовым расстоянием 3 или 4.
2. Разработать приложение для кодирования/декодирования двоичной информации итеративным кодом с различной относительной избыточностью кодовых слов.

3	24	4 3 3 6	6 8 ~~3~~ 2 2	- - 4 2	2; 3 2; 3 2; 3; 4; 5 2; 3; 4; 5

```
import itertools, random, math
DEFAULT = "10110010101100101010110010"

def bits(s):
    if not s or any(c not in "01" for c in s): raise ValueError("Нужно ввести двоичное
слово из 0 и 1.")
    return [int(c) for c in s]

def hm(k, ext=False):
    r = 0
    while 2**r < k + r + 1: r += 1
    n = k + r
    H = [[(j >> i) & 1 for j in range(1, n + 1)] + ([0] if ext else []) for i in range(r)]
    if ext: H.append([1] * (n + 1))
    return r, H

def show_matrix(H):
    for row in H: print(" ".join(map(str, row)))

def hamming():
    word = input("Информационное слово (не менее 16 бит, Enter — пример): ").strip() or
DEFAULT
    x = bits(word)
    if len(x) < 16: raise ValueError("Длина слова должна быть не меньше 16 бит.")
    d = int(input("Минимальное расстояние dmin (3 или 4): ") or "4")
    if d not in (3, 4): raise ValueError("dmin должно быть 3 или 4.")
    ext = d == 4; k = len(x); r, H = hm(k, ext); n = k + r
    code = [0] * n; data = [p for p in range(1, n + 1) if p & (p - 1)]
    for p, b in zip(data, x): code[p - 1] = b
    for p in (1 << i for i in range(r)): code[p - 1] = sum(code[j - 1] for j in range(1, n
+ 1) if j & p) % 2
    if ext: code.append(sum(code) % 2)
    print(f"k={k}, r={r}, n={len(code)}, избыточность={(len(code)-k)/k:.3f},
скорость={k/len(code):.3f}")
    print("Проверочная матрица H:"); show_matrix(H); print("Кодовое слово:",
"".join(map(str, code)))
    for errors in (0, 1, 2):
        y = code[:]; pos = random.sample(range(len(y)), errors)
        for p in pos: y[p] ^= 1
        s = 0
        for i, b in enumerate(y[:n], 1):
            if b: s ^= i
        odd = sum(y) % 2; status = "ошибок не обнаружено"
        if ext:
            if s and odd and s <= n: y[s - 1] ^= 1; status = f"исправлена ошибка в позиции
{s}"
            elif not s and odd: y[-1] ^= 1; status = "исправлена ошибка общего бита
четности"
            elif s and not odd: status = "обнаружена двойная ошибка, исправление не
```

```
выполнялось"
        elif s and s <= n: y[s - 1] ^= 1; status = f"синдром {s}, попытка исправления
позиции {s}"
        decoded = "".join(str(y[p - 1]) for p in data)
        print(f"Ошибок задано: {errors}; позиции: {[p+1 for p in pos]}; синдром={s};
{status}; восстановлено={y == code}; данные={decoded}")

def matrices():
    for k in (4, 6, 8, 9, 10, 15, 16):
        print(f"\n=== k={k}: простая четность, H = [1 ... 1] длины {k+1} ===")
        print(" ".join(["1"] * (k + 1)))
        for d in (3, 4):
            r, H = hm(k, d == 4)
            print(f"dmin={d}, r={r}, n={k+r+(d==4)}:"); show_matrix(H)

def syndrome(y, checks):
    return sum((sum(y[p] for p in chk) % 2) << j for j, chk in enumerate(checks))

def iterative():
    dims = ((4, 6), (3, 8), (3, 2, 4), (6, 2, 2))
    for i, d in enumerate(dims, 1): print(f"{i}. {'×'.join(map(str,d))}")
    shape = dims[int(input("Матрица варианта 3 (1-4): ") or "1") - 1]
    groups = int(input(f"Группы проверок ({'2 или 3' if len(shape)==2 else '2-5'}): ") or
"2")
    if groups not in ((2, 3) if len(shape) == 2 else (2, 3, 4, 5)): raise
ValueError("Недопустимое число групп.")
    word = (input("24 бита данных (Enter — пример): ").strip() or DEFAULT); x = bits(word)
    if len(x) != 24: raise ValueError("Для варианта 3 нужно ровно 24 бита.")
    cap = math.prod(shape); data = x + [0] * (cap - len(x)); code = data[:]; checks = []
    def add(line):
        p = len(code); code.append(sum(code[i] for i in line) % 2); checks.append(line +
[p])
    coords = list(itertools.product(*(range(n) for n in shape)))
    for ax in (0, 1):
        lines = {}
        for i, c in enumerate(coords): lines.setdefault(c[:ax] + c[ax+1:], []).append(i)
        for line in lines.values(): add(line)
    if groups >= 3:
        rows, cols = shape[:2]; layers = shape[2] if len(shape) == 3 else 1
        for z in range(layers):
            for step in (1, -1):
                for r in range(rows):
                    for c in range(cols):
                        if r and 0 <= c-step < cols: continue
                        line = []; rr, cc = r, c
                        while rr < rows and 0 <= cc < cols:
                            line.append((rr*cols+cc)*layers+z if len(shape)==3 else
rr*cols+cc)
                            rr += 1; cc += step
                        if len(line) > 1: add(line)
    if len(shape) == 3 and groups >= 4:
        lines = {}
        for i, c in enumerate(coords): lines.setdefault((c[0], c[1]), []).append(i)
        for line in lines.values(): add(line)
    if len(shape) == 3 and groups == 5: add(list(range(len(code))))
    masks = [0] * len(code)
    for j, chk in enumerate(checks):
        for p in chk: masks[p] |= 1 << j
    print(f"Данные: {''.join(map(str,x))}; матрица: {'×'.join(map(str,shape))}; заполнение
нулями: {cap-24}")
    print(f"Проверочных битов: {len(code)-cap}; кодовое слово: {''.join(map(str,code))}")
    print("Формирую таблицу синдромов для комбинаций до 3 ошибок...")
    lookup = {0: (0, ())}
    for w in (1, 2, 3):
        for comb in itertools.combinations(range(len(code)), w):
            s = 0
            for p in comb: s ^= masks[p]
            old = lookup.get(s)
            if old is None: lookup[s] = (w, comb)
            elif old[0] == w and old[1] is not None and old[1] != comb: lookup[s] = (w,
None)
    def correct(y):
        s = syndrome(y, checks); item = lookup.get(s)
        if s and item and item[1] is not None:
            for p in item[1]: y[p] ^= 1
        return y, s
```

```
    errors = int(input(f"Сколько ошибок внести (0-{len(code)}): ") or "1")
    y = code[:]; pos = random.sample(range(len(y)), errors)
    for p in pos: y[p] ^= 1
    received = y[:]; fixed, s = correct(y)
    print(f"Ошибочные позиции (с 1): {[p+1 for p in pos]}; синдром={s}; исправлено={fixed
== code}")
    print("Принятое слово:  ", ''.join(map(str, received)))
    print("После декодирования:", ''.join(map(str, fixed)))
    print("Информационная часть:", ''.join(map(str, fixed[:24])))
    ns = {}
    for e in (1, 2, 3):
        ns[e] = 0
        for _ in range(100):
            trial = code[:]
            for p in random.sample(range(len(code)), e): trial[p] ^= 1
            ns[e] += correct(trial)[0] == code
    print(f"Успешное исправление за 100 испытаний: N1={ns[1]}, N2={ns[2]}, N3={ns[3]};
N2/N1={ns[2]/ns[1] if ns[1] else 0:.3f}; N3/N1={ns[3]/ns[1] if ns[1] else 0:.3f}")

if __name__ == "__main__":
    print("Лабораторная №2 — вариант 3\n1. Код Хэмминга\n2. Матрицы Хэмминга\n3.
Итеративный код")
    try: {"1": hamming, "2": matrices, "3": iterative}[input("Выберите: ").strip()]()
    except (KeyError, ValueError) as e: print("Ошибка:", e)
```

```
PS C:\Users\VivoBook\Desktop\РАГОЗ\Latish\KiSB-2026-autumn> & C:\Users\VivoBook\AppData\Local\Programs\Python\Python312\python.exe c:/Users/VivoBook/Desktop/РАГОЗ/
h/KiSB-2026-autumn/Ezepchuk/lab2/src/lab2.py
Лабораторная №2 — вариант 3
1. Код Хэмминга
2. Матрицы Хэмминга
3. Итеративный код
Выберите: 1
Информационное слово (не менее 16 бит, Enter — пример):
Минимальное расстояние dmin (3 или 4): 4
k=24, r=5, n=30, избыточность=0.250, скорость=0.800
Проверочная матрица H:
1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0 1 0
0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0 0 1 1 0
0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1 1 1 1 0 0 0 0 1
0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 1 1 1 1 1
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 0
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
Кодовое слово: 10110110001010101001010110010 0
Ошибок задано: 0; позиции: []; синдром=0; ошибок не обнаружено; восстановлено=True; данные=10110010101100101011001 0
Ошибок задано: 1; позиции: [20]; синдром=20; исправлена ошибка в позиции 20; восстановлено=True; данные=10110010101100101011001 0
Ошибок задано: 2; позиции: [22, 16]; синдром=6; обнаружена двойная ошибка, исправление не выполнялось; восстановлено=False; данные=10110010101100100011001 0
PS C:\Users\VivoBook\Desktop\РАГОЗ\Latish\KiSB-2026-autumn>
PS C:\Users\VivoBook\Desktop\РАГОЗ\Latish\KiSB-2026-autumn> & C:\Users\VivoBook\AppData\Local\Programs\Python\Python312\python.exe c:/Users/VivoBook/Desktop/РАГОЗ/
h/KiSB-2026-autumn/Ezepchuk/lab2/src/lab2.py
Лабораторная №2 — вариант 3
1. Код Хэмминга
2. Матрицы Хэмминга
3. Итеративный код
Выберите: 3
1. 4×6
2. 3×8
3. 3×2×4
4. 6×2×2
Матрица варианта 3 (1-4): 3
Группы проверок (2-5): 2
24 бита данных (Enter — пример):
Данные: 101100101011001010110010; матрица: 3×2×4; заполнение нулями: 0
Проверочных битов: 20; кодовое слово: 10110010101100101011001010110010100110011001
Формирую таблицу синдромов для комбинаций до 3 ошибок...
Сколько ошибок внести (0-44): 2
Ошибочные позиции (с 1): [29, 36]; синдром=2064; исправлено=True
Принятое слово:   10110010101100101011001010111010100010011001
После декодирования: 10110010101100101011001010110010100110011001
Информационная часть: 101100101011001010110010
Успешное исправление за 100 испытаний: N1=100, N2=82, N3=34; N2/N1=0.820; N3/N1=0.340
```

Вывод: приобрел практические навыки кодирования декодирования двоичных данных.